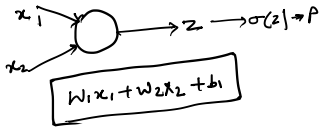
N-Layer Neural Networks: Learning Complex Patterns

Thursday, 24 September 2026 12:36 AM

One line: what's the relationship between a single neuron with sigmoid activation and logistic regression?

They are same.



Why does a hidden layer need a non-linear activation at all?

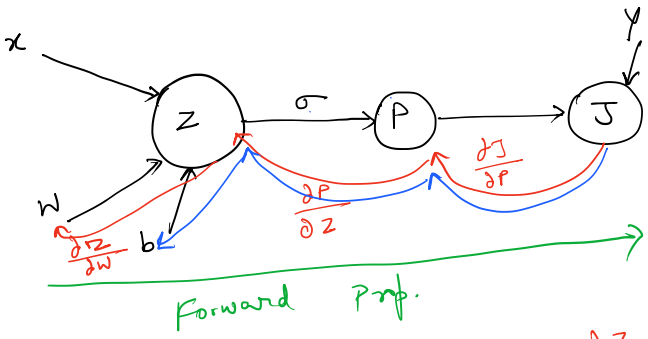
To learn complex / non linear patterns

One sentence: what causes vanishing gradients?

Very small gradients due to deep networks. model is not able to learn.

$(0.9)^{50} \to$ very small value

0-1



$z = w^T x + b$

$p = \sigma(z)$

$J = p - y$

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial p} \times \frac{\partial p}{\partial z} \times \frac{\partial z}{\partial w}$$

$z = w \cdot x + b.$

$$w^{new} = w^{old} - ? \frac{\partial J}{\partial w}$$

$$\frac{\partial J}{\partial w} = (p - y) \cdot x$$

derivation in next class

$$\frac{\partial J}{\partial b} = \frac{\partial J}{\partial p} \times \frac{\partial p}{\partial z} \times \frac{\partial z}{\partial b}$$

$(p - y) \cdot 1$

$$\frac{\partial J}{\partial b} = p - y$$

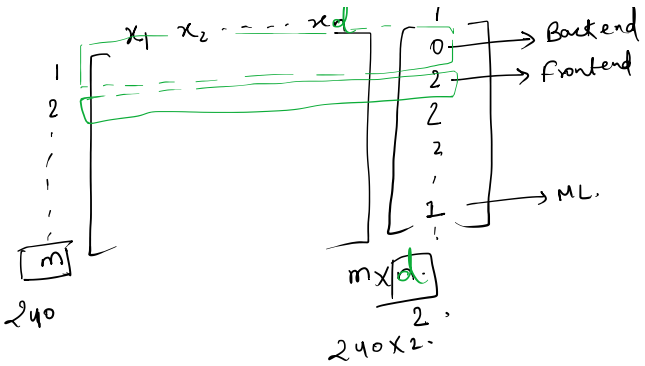
A single neuron computes $z = w \cdot x + b$ and then applies an activation function to z to produce its output.

If the activation function is sigmoid, which classical model does this neuron become functionally identical to?

Linear Regression, do we use sigmoid?

✓ Logistic Regression uses Sigmoid.

$X =$

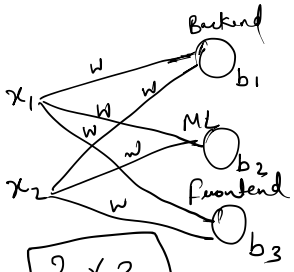


240 data points → Y (0, 1, 2)

2 features.

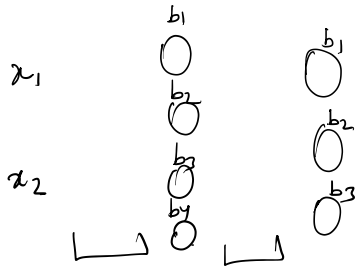
$Y =$ Backend (0), ML (1), Frontend (2).

Backend



$W =$ 2×3

$b =$ $[1 \times 3]$



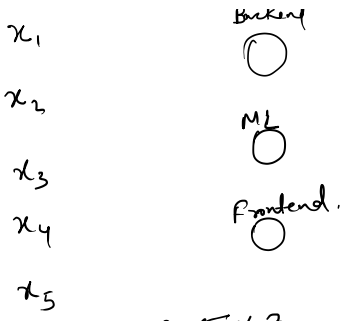
$W = 2 \times 4$ $\quad W = 4 \times 3$

$b = [1 \times 4]$ $\quad b = 1 \times 3$

$W \cdot X + b$

$[4 \times 3] + [1 \times 3]$ ⟹ We use broadcasting.

$b \rightarrow [0.02 \quad 0.01 \quad 0.029]$



$W = ?$ 5×3

$b = ?$ 1×3

3 classes.

DP 1 → class 0 → [1 0 0]

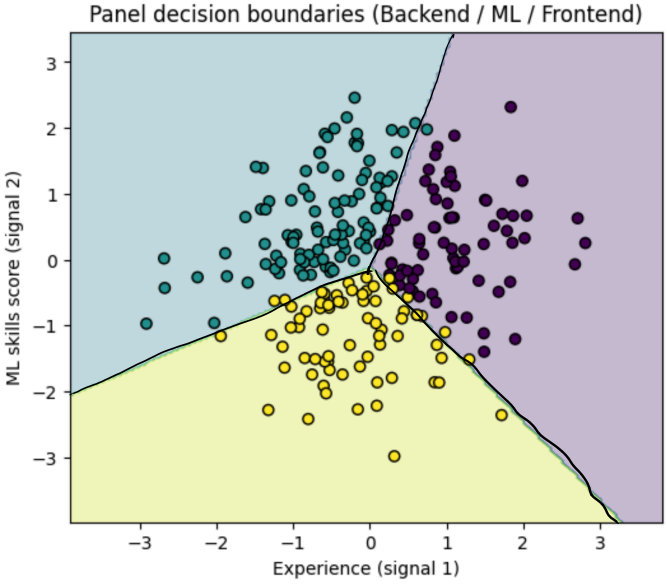
DP 2 → class 1 → [0 1 0]

DP 3 → class 2 → [0 0 1]

⋮ ⋮

DP 240 → class 1 → [0 1 0]

(240 x 3)

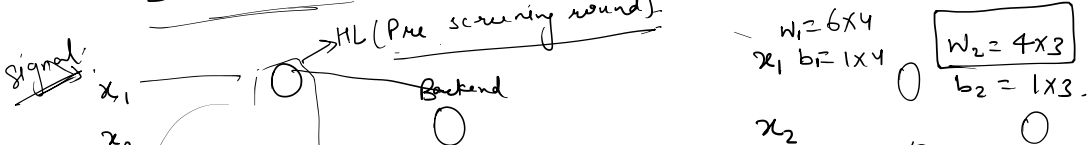


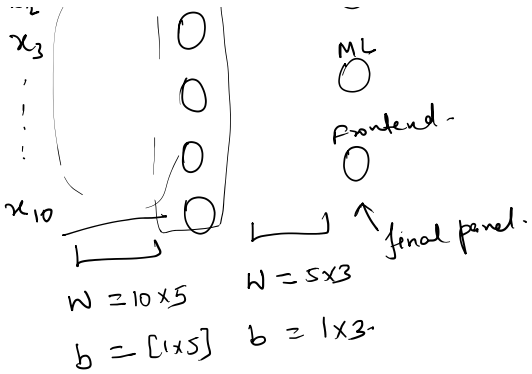
Linear Decision Boundary.

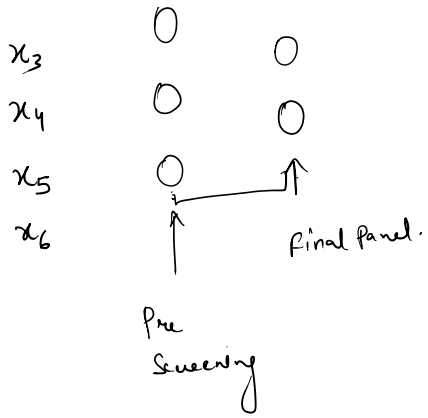
10 Signals. → 10 features.

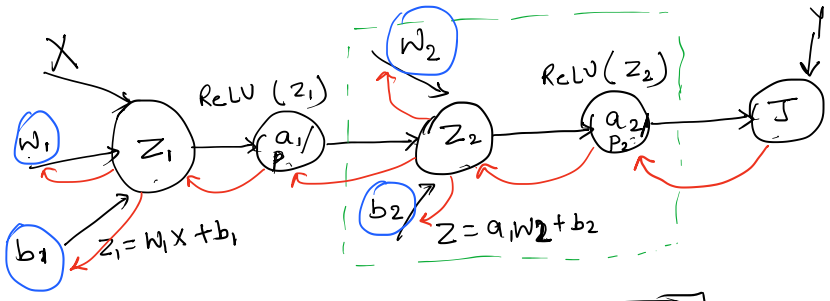
5 Units → 5 neurons. in the HL

3 tracks → 3 neurons in output layer.









$$\frac{\partial J}{\partial W_2} = \frac{\partial J}{\partial a_2} \times \frac{\partial a_2}{\partial z_2} \times \frac{\partial z_2}{\partial W_2} \leftarrow (b_2).$$

$$\frac{\partial J}{\partial W_1} = \frac{\partial J}{\partial a_2} \times \frac{\partial a_2}{\partial z_2} \times \frac{\partial z_2}{\partial a_1} \times \frac{\partial a_1}{\partial z_1} \times \frac{\partial z_1}{\partial W_1} \quad (b_1)$$

$$W^{2\,new} = W^{2\,old} - \eta \frac{\partial J}{\partial W_2}$$

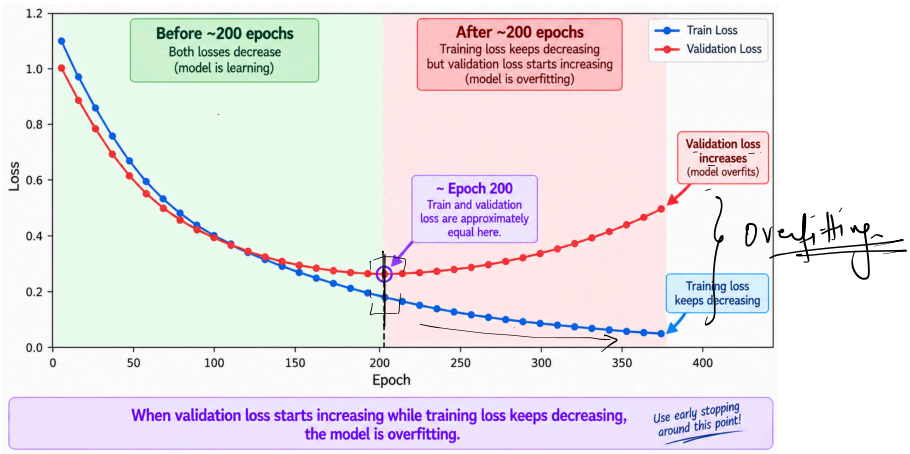
$$W^{1\,new} = W^{1\,old} - \eta \frac{\partial J}{\partial W_1}$$

```
epoch    0 | train_loss 3.8843 | val_loss 2.1092
epoch  200 | train_loss 0.0242 | val_loss 0.1388
epoch  400 | train_loss 0.0139 | val_loss 0.1675
epoch  600 | train_loss 0.0098 | val_loss 0.1886
epoch  800 | train_loss 0.0077 | val_loss 0.2057
epoch 1000 | train_loss 0.0063 | val_loss 0.2197
epoch 1200 | train_loss 0.0054 | val_loss 0.2311
epoch 1400 | train_loss 0.0047 | val_loss 0.2408
epoch 1600 | train_loss 0.0042 | val_loss 0.2496
epoch 1800 | train_loss 0.0038 | val_loss 0.2574
epoch 2000 | train_loss 0.0034 | val_loss 0.2645
epoch 2200 | train_loss 0.0031 | val_loss 0.2708
epoch 2400 | train_loss 0.0029 | val_loss 0.2767
epoch 2600 | train_loss 0.0027 | val_loss 0.2821
epoch 2800 | train_loss 0.0025 | val_loss 0.2871
epoch 3000 | train_loss 0.0024 | val_loss 0.2917
```

look good.

val loss increased train loss dropping.

Model Starts Overfitting



Q. How do we handle Overfitting?

- Use a simpler model
- Regularization.
- Early stopping. → Don't overtrain
- More Data

Scenario: I have 500 training examples, 12 input features and 4 output classes. Suggest 2-3 reasonable architectures, with reasoning.

500 data points.
Use simpler Models.

